$python-learning-explainer

第 29 章：综合实践：可取消的最小 Agent 工具执行器

我的学习背景与目标：

- 我已经掌握 Python 基础：容器、控制流、函数与参数、一等函数、类、模块，以及文件与 JSON 的基本处理。
- 我接下来要读懂 Agent 框架并开发 Agent，重点是 LLM、MCP、HTTP、数据库等 I/O，以及多个 tool 的并发执行。
- 请直接使用 Python 的概念讲解，不再加入 Java 对照。不要依赖其他对话历史，也不要把"基础课代码里出现过某语法"当作我已经深入理解它。

本次工作方式与交付：

1. 先读取适用的 AGENTS.md、pyproject.toml、目标章节已有文件和直接相关的前置材料。以实际存在的文件为准；前置章节不存在时，在本章补一个最小前置说明，不要顺便生成其他章节。
2. 项目规划时使用 Python 3.14；请核对当前项目约束与实际解释器。需要区分的版本语义应注明，第三方库请检查实际版本与官方文档，附上直接支持关键结论的链接。
3. 请直接在下述目标目录创建或强化 README.md 和 example.py；必要时拆出少量辅助模块、数据或测试。保留已有正确内容，不修改前 10 章和 skill 本身。目录编号不是合法导入名时，不要用修改 sys.path 的办法拼接跨章依赖。
4. README 要讲清问题、运行过程、最小正确例子、实际场景、反例原因、选择标准和练习；注释重点解释为什么这样写。每次只完成本章，前置概念只补必要部分，延伸内容标为选学。
5. 示例应有清晰入口、中文分段输出和可观察结果，默认在本机运行，不要求真实 API Key 或付费服务。实际 HTTP 可使用本地服务，纯原理实验用可控替身，并明确替身没有覆盖的真实行为。
6. 需要依赖时沿用项目已有管理方式，只添加本章所需依赖，区分运行与开发依赖，提供从仓库根目录独立运行本章的命令。现有 run_all.py 仅支持前 10 章，不要假定新章已经接入。不要重建已有虚拟环境、覆盖项目配置或为了检查而安装无关工具。
7. 反例应可控、有限时并正确清理，不能让入口永久挂起或污染后续示例。并发实验用事件或可观察状态验证关键顺序，不依赖精确 sleep 时长与固定调度顺序。
8. 用符合项目版本的解释器实际运行完整示例，执行已配置且可用的相关检查。分别说明已验证结果与未验证限制，不能把静态检查说成运行验证。
9. 每章提供 3～5 个递进练习，给要求、输入和验收标准，不直接附上全部答案。最后简要列出文件、运行命令、验证结果和仍需我掌握的关键点。

本章目标目录：java_to_python/chapters/29_mini_agent_runtime/

前置知识与承接：第 11～27 章的核心知识；第 28 章服务化可选，不作为运行本章的必需条件。

学完应达到：能把类型、装饰器、异步任务和资源管理组合成一个可测试的最小 Agent 循环。

本章以"文档与记录查询助手"为一个完整教学项目，逐步实现，不先抛出整套框架代码。

9. 每章提供 3～5 个递进练习，给要求、输入和验收标准，不直接附上全部答案。最后简要列出文件、运行命令、验证结果和仍需我掌握的关键点。

本章目标目录：java_to_python/chapters/29_mini_agent_runtime/

前置知识与承接：第 11～27 章的核心知识；第 28 章服务化可选，不作为运行本章的必需条件。

学完应达到：能把类型、装饰器、异步任务和资源管理组合成一个可测试的最小 Agent 循环。

本章以"文档与记录查询助手"为一个完整教学项目，逐步实现，不先抛出整套框架代码。

数据与边界：

- 明确 UserRequest、ToolCall、ToolResult、AgentEvent、RunState 的用途；外部输入用 Pydantic 验证，内部状态按实际需要选择 dataclass。
- ToolCall 包含稳定的 call_id、工具名和参数；并发完成顺序改变时，结果仍能准确匹配原调用。
- 通过装饰器注册少量明确工具，拒绝未知工具和不合法参数；不执行任意生成的 Python 代码。
- 用协议表达模型适配器和必要的工具依赖，避免把 SDK 对象散落在整个运行循环里。

运行主线：
用户输入 → 模型给出工具请求 → 校验 → 执行独立工具 → 将结果反馈给模型 → 最终回答。

- 使用可控 FakeLLM 按预设步骤返回工具调用与最终答案；清楚说明这是模型替身，不能据此证明真实推理质量。
- 默认至少 3 个工具：本地文档检索、本地 HTTP 状态查询、临时 SQLite 记录查询；全部本机可运行。
- 只并发调度相互独立的调用；有依赖的下一轮工具等待前一轮结果，不把所有 tool call 都盲目并行。
- 提供明确的 fail-fast 与允许部分业务失败的策略，统一错误结果结构，同时保留取消与未知程序异常。
- 有整体截止时间、单工具预算、并发上限和最大模型调用轮数；重试不额外获得无限预算。
- 同时提供最终结果与 async iterator 事件流，能显示 tool-start、tool-result 和结束状态；提前停止消费时清理执行中的工作。
- 谁拥有 HTTP 客户端、会话、TaskGroup 和事件生成器必须明确；ContextVar 用于运行日志标识。

交付与验收：
将 models、tools、runtime、fakes、tests 等按确有必要的职责拆分，example.py 保留清晰入口与场景选择。
循序提供最小顺序版、并发版和完整失败处理版的讲解，避免为保留所有版本复制大量实现。
运行成功、参数错误、未知工具、部分失败、整体超时、用户取消、超出最大轮数和提前退出事件流等测试。
如果输出使用 Queue，必须有容量、结束协议和异常传递设计；也可以采用更简单且满足需求的结构。
最终用一张组件关系图解释每个 Python 技能落在哪里，再指出下一章哪些职责将交给真实框架。

边界：
这是用来理解框架机制的教学运行器，不扩展成生产级 Agent 框架。
不增加向量数据库、多 Agent 分工、长期记忆、复杂规划或外部部署；真实模型调用作为显式可选入口。

Python 基础之后：Agent 开发学习路线与逐章 Prompt

这里规划了 **20 个部分，对应未来的第 11～30 章。**目标是从"能写 Python"走到"能读懂 Agent 框架，并处理真实工具调用的并发、失败与资源生命周期"。

当前交付的是路线和生成课程用的 prompt。第 11～30 章的教程与代码将在后续对话中逐章创建；本目录中的文件本身不代表相应课程已经实现。

怎样使用

1. 在下面的目录中打开要学的一章。
2. 复制文件全文，从第一行 `$python-learning-explainer` 开始，粘贴到这个仓库的新对话。
3. 该 prompt 会要求模型读取必要的项目上下文，并在对应的 `java_to_python/chapters/编号_主题/` 中创建 README.md、example.py 及必要辅助文件。
4. 阅读讲解、预测实验结果、运行代码，再完成练习。遇到不理解的运行过程，在该章对话中继续追问。

每份 prompt 都能独立复制。 学习背景、Python 版本核验、教学方式、输出路径和验证要求已经放进每份文件，不需要另外拼接一段"公共提示词"。独立复制不等于没有知识前置：每份文件也说明了应该先会什么。

如果只想先学而暂时不写文件，可以在复制后追加："本次仅在回复中教学，暂不创建或修改文件。"当前对话的明确要求优先于 prompt 中的默认交付方式。

为什么这样拆

前 10 章已经覆盖容器、函数、对象、模块，以及 JSON 解析、业务校验与 with 文件管理。第 11～12 章因此集中于解释器与依赖的工程关系、HTTP 请求和外部数据边界，不再重复 pickle、CSV 或基础文件语法。

第 13～18 章补上读框架代码需要的语言能力：类型、数据模型、协议与泛型、装饰器、生成器和资源协议。这里要分别理解静态类型提示和运行时输入验证；Python 默认不强制执行函数与变量注解，Pydantic 承担的是另一层职责。Python typing 文档、Pydantic 模型文档

第 19～24 章是整条路线的重点。 先解释 coroutine 和调度，再学习 Task，然后单独处理取消与超时，之后进入 TaskGroup、并发控制和异步流。把取消放在 TaskGroup 前面，是因为任务组的失败处理本身就涉及取消兄弟任务和等待清理；不能只学会一个 async with 写法。Python 3.14 coroutine 与 Task 文档

第 25～28 章把这些机制应用到 HTTP、数据库、上下文、测试和 Web 服务。HTTPX 客户端复用与流关闭、FastAPI 应用生命周期都涉及资源所有权，因此安排在异步资源章节之后。HTTPX 异步客户端文档、FastAPI lifespan 文档

第 29 章用小型运行器串起已学知识，第 30 章迁移到真实框架并阅读实际调用链。未指定框架时暂定 PydanticAI，主要为了衔接类型模型、函数工具与异步运行；其测试模型与 MCP 客户端文档也提供了本地验证的切入点。开始第 30 章时仍需核对当时版本与兼容性，也可直接替换为你选定的框架。PydanticAI 测试文档、MCP 客户端文档

Ready

逐章目录

点击主题即可打开完整 prompt。所有新章节都使用 Python 讲解与示例，不新增 Java 对照程序。

章	可复制的 Prompt	阶段	学完应达到
11	虚拟环境、pip 与 uv 项目管理	实用基础	能从一个干净环境安装依赖并运行项目，定位"终端能 import、IDE 不能"的原因。
12	使用 requests 调用 API 与处理 JSON 边界	实用基础	能把一次 API 调用拆成请求构造、传输、状态判断、JSON 解析和数据校验。
13	类型注解、TypedDict 与静态检查	语言与抽象	能为工具输入和返回值描述准确的类型，并区分静态约束与运行时验证。
14	dataclass、Pydantic 与结构化工具数据	语言与抽象	能为 Agent 工具参数、结果和内部状态选择合适的数据结构。
15	Protocol、ABC 与实用泛型	语言与抽象	能定义可替换的工具接口，读懂输入与输出类型之间的泛型关系。
16	装饰器、闭包与工具注册	语言与抽象	能解释 @tool 的执行时机，写出保留元信息与类型关系的同步装饰器。
17	迭代器、生成器与惰性数据流	语言与抽象	能判断数据何时产生、何时被消费，为后续 LLM 流式输出建立基础。
18	上下文管理器与资源生命周期	语言与抽象	能给工具会话和其他资源定义清晰的获取、使用、清理边界。
19	asyncio：coroutine、await 与事件循环	asyncio 核心	能从运行过程解释 coroutine 与 await，分清异步语法和实际并发。
20	Task 生命周期与并发结果收集	asyncio 核心	能启动、持有、等待和回收任务，按需要选择结果收集方式。
21	取消、超时与截止时间	asyncio 核心	能设计一条可取消的工具调用链，解释超时对任务和外部 I/O 的实际影响。
22	TaskGroup、结构化并发与异常组	asyncio 核心	能为一次 Agent 请求建立任务作用域，并明确选择整体失败或允许部分成功。
23	并发上限、Queue、背压与共享状态	asyncio 核心	能让大量工具任务在有限资源下运行，并正确停止生产者与消费者。
24	异步生成器、流式消费与异步资源管理	asyncio 核心	能安全消费 LLM 风格的事件流，在提前退出或取消时关闭流和会话。
25	HTTPX、可靠请求与同步 I/O 边界	I/O 与工程实践	能把可控异步实验迁移到实际 HTTP 与数据库调用，正确处理连接复用、重试和阻塞。
26	contextvars、日志与异步装饰器	I/O 与工程实践	能为并发请求传递正确的运行上下文，并记录覆盖真实异步执行过程的日志。
27	pytest、异步测试与故障验证	I/O 与工程实践	能写稳定、离线的异步测试，证明错误路径与资源清理确实符合设计。

▼ Python
发学习
怎样
为什
逐章
学习
每份
知识
贯穿
仓库

26	contextvars、台怨与异步装饰器	与工程实践	能为并发请求传递正确的运行上下文，并记录覆盖真实异步执行边界的日怨。
27	pytest、异步测试与故障验证	I/O 与工程实践	能写稳定、离线的异步测试，证明错误路径与资源清理确实符合设计。
28	FastAPI、依赖与异步服务生命周期	I/O 与工程实践	能把工具编排暴露为可验证的 API，并理解 Web 请求与后台任务的生命周期边界。
29	综合实践：可取消的最小 Agent 工具执行器	Agent 实战	能把类型、装饰器、异步任务和资源管理组合成一个可测试的最小 Agent 循环。
30	真实 Agent 框架阅读、迁移与 MCP	Agent 实战	能沿真实框架的调用链阅读代码，把已有工具迁移进去，并接入一个真实本地 MCP 服务。

学习顺序与投入重点

默认按 11 → 30 学习，有以下调整空间：

- **第 11～12 章可以快速过。** 如果已经能解释解释器与 pip 的对应关系，能重建依赖环境，并区分传输、HTTP、JSON 与业务错误，可直接做验收练习。
- **第 13～18 章按需补深度。** 类型、数据模型和装饰器应掌握；第 15 章泛型的复杂变型推导只作选学。第 17～18 章用于支撑后续异步流与会话清理。
- **第 19～24 章建议连续学习。** 不把 coroutine、Task、TaskGroup 压缩为同义词，也不跳过取消与失败路径实验。
- **第 25～27 章用于检验能否写可靠 I/O 代码。** 真实传输和可控替身分开验证；测试尤其关注取消、并发上限与资源释放。
- **第 28 章可在需要服务化时学习。** 若想先进入框架，可从第 27 章进入第 29～30 章，再回来做 FastAPI。
- **第 29～30 章逐步组合。** 第 29 章关注运行机制，第 30 章关注真实框架接管了什么，以及 MCP 会话如何创建与关闭。

20 个部分不是固定 20 天。一次对话可以生成一个完整章节；学习时可以只完成一个实验再继续。第 25、29、30 章包含多个递进步骤，适合分多轮阅读和练习。

每份 Prompt 为什么不只是知识点清单

每份文件同时提供：

- 已会什么、还不应假定会什么，以及与前章的关系。
- 本章要解决的实际问题和必须解释的运行行为。
- 最小实验、Agent 场景、常见错误与可观察证据。
- 明确的章节边界，防止一个问题扩展成整套框架。
- 具体输出目录、运行要求、练习和验收标准。

没有额外的历史 context 也可以开课：prompt 已经把“学习问题”组织成了输入材料。后续模型仍需读取项目文件与对应官方资料，不能把 prompt 中的知识点名称当成已经完成的讲解或验证。

每份文件同时提供：

- 已会什么、还不应假定会什么，以及与前章的关系。
- 本章要解决的实际问题和必须解释的运行行为。
- 最小实验、Agent 场景、常见错误与可观察证据。
- 明确的章节边界，防止一个问题扩展成整套框架。
- 具体输出目录、运行要求、练习和验收标准。

没有额外的历史 context 也可以开课：prompt 已经把"学习问题"组织成了输入材料。后续模型仍需读取项目文件与对应官方资料，不能把 prompt 中的知识点名称当成已经完成的讲解或验证。

贯穿路线的业务场景

统一使用一个小型"文档与记录查询助手"，逐步引入：

1. 本地文档检索 search_docs。
2. 本地 HTTP 状态查询 fetch_status。
3. 临时 SQLite 记录查询 lookup_record。
4. 可控 FakeLLM 返回工具请求与最终回答。
5. 最后一章用真实框架和一个真实本地 MCP 服务接入这些工具。

不同章节可以用更小的独立样例解释单条规则，不必强行引用前章内部代码。所有基础实验默认不需要付费服务或真实 API Key。FakeLLM 用于验证调用链，不能据此声称验证了真实模型推理；普通函数替身也不能冒充 MCP 协议集成。

仓库约定与版本

规划依据是当前仓库实际文件： `pyproject.toml` 要求 Python >= 3.14，Pyright 与 Ruff 目标版本为 3.14，本机项目解释器验证为 3.14.7。未来生成章节时应重新核对，而不是把规划时版本当成永远不变。

现有 `java_to_python/run_all.py` 只登记第 1～10 章，而且假设章节有 Java 程序。新增课程先提供各章独立 Python 运行命令；这份规划没有把未来章节提前接入现有运行器，也没有改动现有课程与依赖环境。后续若要统一执行全部课程，应单独适配纯 Python 章节和可选第三方依赖。

依赖学习先分清 venv/pip，再引入 uv 的声明、锁定与同步工作流。进入后续章节时沿用当时已选定的项目方式，不为每个包重新设计一套管理规则。uv 项目文档

这些文件是未来对话的用户 prompt，供主动复制使用；它们不是自动适用于整个仓库的 AGENTS.md 或 skill 修改。

$python-learning-explainer

第 30 章：真实 Agent 框架阅读、迁移与 MCP

我的学习背景与目标：

- 我已经掌握 Python 基础：容器、控制流、函数与参数、一等函数、类、模块，以及文件与·JSON 的基本处理。
- 我接下来要读懂 Agent 框架并开发 Agent，重点是 LLM、MCP、HTTP、数据库等 I/O，以及多个 tool 的并发执行。
- 请直接使用 Python 的概念讲解，不再加入 Java 对照。不要依赖其他对话历史，也不要把"基础课代码里出现过某语法"当作我已经深入理解它。

本次工作方式与交付：

1. 先读取适用的 AGENTS.md、pyproject.toml、目标章节已有文件和直接相关的前置材料。以实际存在的文件为准；前置章节不存在时，在本章补一个最小前置说明，不要顺便生成其他章节。
2. 项目规划时使用 Python 3.14；请核对当前项目约束与实际解释器。需要区分的版本语义应注明，第三方库请检查实际版本与官方文档，附上直接支持关键结论的链接。
3. 请直接在下述目标目录创建或强化 README.md 和 example.py；必要时拆出少量辅助模块、数据或测试。保留已有正确内容，不修改前 10 章和 skill 本身。目录编号不是合法导入名时，不要用修改 sys.path 的办法拼接跨章依赖。
4. README 要讲清问题、运行过程、最小正确例子、实际场景、反例原因、选择标准和练习；注释重点解释为什么这样写。每次只完成本章，前置概念只补必要部分，延伸内容标为选学。
5. 示例应有清晰入口、中文分段输出和可观察结果，默认在本机运行，不要求真实 API Key 或付费服务。实际 HTTP 可使用本地服务，纯原理实验用可控替身，并明确替身没有覆盖的真实行为。
6. 需要依赖时沿用项目已有管理方式，只添加本章所需依赖，区分运行与开发依赖，提供从仓库根目录独立运行本章的命令。现有 run_all.py 仅支持前 10 章，不要假定新章已经接入。不要重建已有虚拟环境、覆盖项目配置或为了检查而安装无关工具。
7. 反例应可控、有限时并正确清理，不能让入口永久挂起或污染后续示例。并发实验用事件或可观察状态验证关键顺序，不依赖精确 sleep 时长与固定调度顺序。
8. 用符合项目版本的解释器实际运行完整示例，执行已配置且可用的相关检查。分别说明已验证结果与未验证限制，不能把静态检查说成运行验证。
9. 每章提供 3～5 个递进练习，给要求、输入和验收标准，不直接附上全部答案。最后简要列出文件、运行命令、验证结果和仍需我掌握的关键点。

本章目标目录：java_to_python/chapters/30_agent_framework_mcp/

前置知识与承接：第 29 章最小工具执行器；重点复用类型模型、协议、装饰器、取消、异步流和资源所有权。

学完应达到：能沿真实框架的调用链阅读代码，把已有工具迁移进去，并接入一个真实本地 MCP 服务。

框架选择与资料：

学完应达到：能沿真实框架的调用链阅读代码，把已有工具迁移进去，并接入一个真实本地 MCP 服务。

框架选择与资料：

- 如果我在本次对话指定框架，按我指定的框架；如果仓库已有明确选择，沿用它；否则本章默认使用 PydanticAI。
- 默认选择只是为了把前面的类型模型、工具函数与异步调用串起来，不展开框架排名，也不同时实现多个框架。
- 开始时核实当前稳定版本与 Python 3.14 兼容性，优先查阅相应版本的官方文档和源码，记录实际依赖版本。不要凭记忆拼接不同版本 API。
- 如果兼容性不满足，明确说明证据和可行方案，保留项目 Python 版本约束，先完成不依赖该冲突的阅读与设计工作。

第一步：运行一个真实框架的最小 Agent

- 使用框架真实的 Agent、工具注册、模型输入输出和异步运行入口；默认采用框架支持的测试模型或可控模型适配器，不要求付费 Key。
- 把第 29 章的 2～3 个工具迁移进去，保持原业务与失败场景，不能只写一段伪装成框架 API 的示例。
- 对照说明哪些职责交给框架、哪些仍由应用负责，例如并发上限、HTTP 生命周期、幂等性和外部副作用。
- 验证多工具执行、结构化输出和流式事件；区分模型在一轮中提出多个调用与框架真正并行执行这些调用。

第二步：沿调用链读代码
选择一条真实链路："调用异步运行入口 → 模型响应 → 参数校验 → 工具执行 → 事件或最终结果"。

- 引用当前版本源码的文件、符号和必要行号，不整仓库漫游。
- 标出 coroutine、Task/任务组、协议、类型变量、装饰器、上下文和资源管理的真实位置；实现若使用 AnyIO 等抽象，说明它与已学概念的关系。
- 实测一个工具异常、外部取消和整体超时，解释实际传播边界；不能把自写执行器的行为当作框架保证。
- 结合一个真实函数签名说明参数与返回值类型，结合一段真实资源代码说明谁负责关闭。

第三步：接入一个本地 MCP 工具

- 先简要区分 Agent 框架、模型 tool calling、MCP host/client/server、传输与会话，避免把它们混成同一个系统。
- 使用当前官方 SDK/框架支持的方式启动一个小型本地 stdio MCP 服务，暴露只读检索或回显工具，完成连接、发现、调用和退出。
- stdio 与 HTTP 传输只做必要比较；主实验选一种，避免同时搭两套服务。
- 管理会话和本地子进程生命周期，演示正常关闭和取消后的清理。说明本地取消是否能停止服务端工作需依据实现验证。
- 只有实际经过 MCP 协议的调用才标为 MCP 集成验证；普通 Python 函数替身仅用于独立单测，不能冒充该步骤完成。

交付与验收：
可按 framework_demo、mcp_demo、tests 拆文件，example.py 提供有限时入口。
给出锁定依赖、完整本地启动步骤、无需付费服务的验证方式、关键源码链接和运行结果。
真实模型调用可作为明确区分的可选路径，记录未执行就没有验证的行为。
我应能亲自沿入口定位工具执行代码，解释用户取消会传到哪里，并说清一次 MCP 调用拥有哪些连接与任务。